Experiment 9: CPM Analysis

Problem 1

Aim: Find critical path: A(3), B(4,A), C(5,A), D(2,B), E(6,C), F(3,D,E).

Procedure: Forward pass ES/EF, backward pass LS/LF, slack=LS-ES, critical=zero slack.

Coding:

```
[1]
import networkx as nx
G = nx.DiGraph()
G.add_weighted_edges_from([('S','A',3),('A','B',4),('A','C',5),('B','D',2),('C','E',6),('D','F',3),('E','F',0)])
print("Duration:", nx.dag_longest_path_length(G, weight='weight'))
print("Critical:", nx.dag_longest_path(G))

Duration: 14
Critical: ['S', 'A', 'C', 'E']
```

Output:

Duration: 14

Critical: ['S', 'A', 'C', 'E']

Problem 2

Aim: Construction: Fnd(5), Walls(10,F), Roof(7,W), Elec(4,W), Paint(3,R,E).

Procedure: Network with parallel electrical path.

Coding:

```
[2]
✓ 0s
G = nx.DiGraph()
G.add_weighted_edges_from([('S','F',5),('F','W',10),('W','R',7),('W','E',4),('R','P',3),('E','P',0)])
print("Duration:", nx.dag_longest_path_length(G, weight='weight'))

Duration: 25
```

Output:

Duration:25

Problem 3

Aim: Software: Req(4), Des(6,R), Code(8,D), Test(5,C), Deploy(3,T).

Procedure: Development pipeline analysis.

Coding:

```
G = nx.DiGraph()
G.add_weighted_edges_from([('S','R',4),('R','D',6),('D','C',8),('C','T',5),('T','P',3)])
print("Duration:", nx.dag_longest_path_length(G, weight='weight'))
print("Critical:", nx.dag_longest_path(G))
```

```
Duration: 26
Critical: ['S', 'R', 'D', 'C', 'T', 'P']
```

Output:

Duration: 26

Critical: ['S', 'R', 'D', 'C', 'T', 'P']

Problem 4

Aim: Manufacturing: Raw(2), Assy(5,R), QC(3,A), Pack(2,Q).

Procedure: Linear production flow.

Coding:

```
[6]
import networkx as nx
G = nx.DiGraph()
G.add_weighted_edges_from([('S','R',2),('R','A',5),('A','Q',3),('Q','P',2)])
print("Duration:", nx.dag_longest_path_length(G, weight='weight'))

print("Critical:", nx.dag_longest_path(G))

Duration: 12
Critical: ['S', 'R', 'A', 'Q', 'P']
```

Output:

Duration: 12

Critical: ['S', 'R', 'A', 'Q', 'P']

Problem 5

Aim: R&D: Lit(1), Hyp(3,L), Exp(7,H), Anal(4,E), Report(2,A).

Procedure: Research with experimental bottleneck.

Coding:

```
G = nx.DiGraph()
G.add_weighted_edges_from([('S','R',1),('R','D',3),('D','C',7),('C','T',4),('T','P',2)])
print("Duration:", nx.dag_longest_path_length(G, weight='weight'))
print("Critical:", nx.dag_longest_path(G))
```

```
Duration: 17
Critical: ['S', 'R', 'D', 'C', 'T', 'P']
```

Output:

Duration: 17

Critical: ['S', 'R', 'D', 'C', 'T', 'P']

Problem 6

Aim: Event: Venue(2), Cater(5,V), Decor(3,V), Prog(4,C,D).

Procedure: Parallel preparation activities.

Coding:

```
G = nx.DiGraph()
G.add_weighted_edges_from([('S','V',2),('V','C',5),('V','D',3),('C','P',4),('D','P',0)])
print("Duration:", nx.dag_longest_path_length(G, weight='weight'))
print("Critical:", nx.dag_longest_path(G))
```

```
Duration: 11
Critical: ['S', 'V', 'C', 'P']
```

Output:

Duration: 11

Critical: ['S', 'V', 'C', 'P']

Problem 7

Problem 10

Aim: Construction with dummies: A3,B4,C5(A,B),D2(E),E3(C),F1(D,E).

Procedure: Dummies enforce precedence without duration.

Coding:

```
G = nx.DiGraph()
G.add_weighted_edges_from([('S','A',3),('S','B',4),('A','D',0),('B','D',0),
 ('D','E',5),('E','F',2),('F','P',1)])
print("Duration:", nx.dag_longest_path_length(G, weight='weight'))
print("Critical:", nx.dag_longest_path(G))
```

```
Duration: 12
Critical: ['S', 'B', 'D', 'E', 'F', 'P']
```

Output:

Duration: 12, Critical: S-B-D-E-F-P